CLASS ASSIGNMENT – 10
ID : 12341550
DATE : 04-11-25

Q1

code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 102  // number of cycles

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int turn = 0; // 0 -> A, 1 -> B, 2 -> C

void *printA(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        while (turn != 0)
            pthread_cond_wait(&cond, &lock);

        printf("A ");
        fflush(stdout);

        turn = 1;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

```
void *printB(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);

        while (turn != 1)
            pthread_cond_wait(&cond, &lock);
        printf("B ");
        fflush(stdout);

        turn = 2;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void *printC(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        while (turn != 2)
            pthread_cond_wait(&cond, &lock);

        printf("C ");
        fflush(stdout);

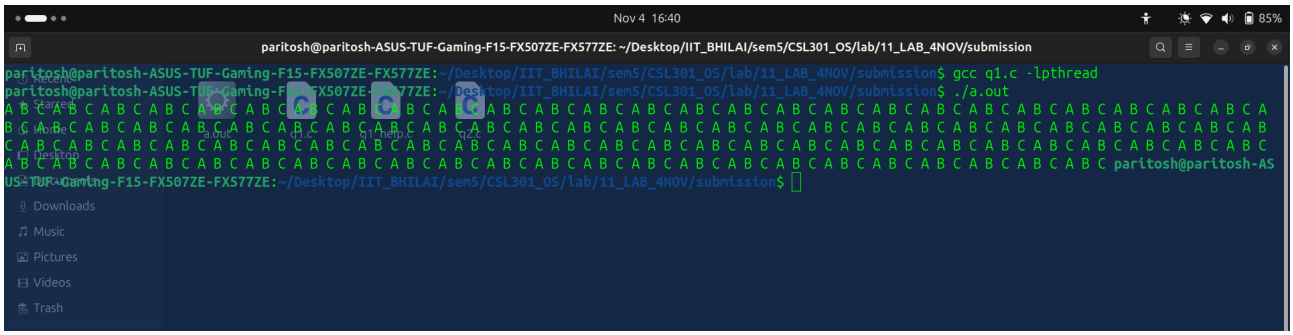
        turn = 0;
        pthread_cond_broadcast(&cond);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t tA, tB, tC;
```

```
    pthread_create(&tA, NULL, printA, NULL);
    pthread_create(&tB, NULL, printB, NULL);
    pthread_create(&tC, NULL, printC, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);
    pthread_join(tC, NULL);
    return 0;
}
```

output:



Explanation:
Switched to condition-variable rotation so exactly one thread prints at a time and order cycles A → B → C.

Q2

code :

```
    #include <stdio.h>
    #include <pthread.h>
    #include <unistd.h>

    pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
    pthread_cond_t can_read = PTHREAD_COND_INITIALIZER;
    pthread_cond_t can_write =
PTHREAD_COND_INITIALIZER;
```

```
int read_count = 0;
int write_count = 0;
int waiting_writers = 0;

void start_read() {
    pthread_mutex_lock(&lock);
    while (write_count > 0 || waiting_writers > 0) {
        pthread_cond_wait(&can_read, &lock);
    }
    read_count++;
    pthread_mutex_unlock(&lock);
}

void end_read() {
    pthread_mutex_lock(&lock);
    read_count--;
    if (read_count == 0)
        pthread_cond_signal(&can_write);
    pthread_mutex_unlock(&lock);
}

void start_write() {
    pthread_mutex_lock(&lock);
    waiting_writers++;
    while (write_count > 0 || read_count > 0) {
        pthread_cond_wait(&can_write, &lock);
    }
    waiting_writers--;
    write_count = 1;
    pthread_mutex_unlock(&lock);
}

void end_write() {
    pthread_mutex_lock(&lock);
```

```
    write_count = 0;
    if (waiting_writers > 0)
        pthread_cond_signal(&can_write);
    else
        pthread_cond_broadcast(&can_read);
    pthread_mutex_unlock(&lock);
}

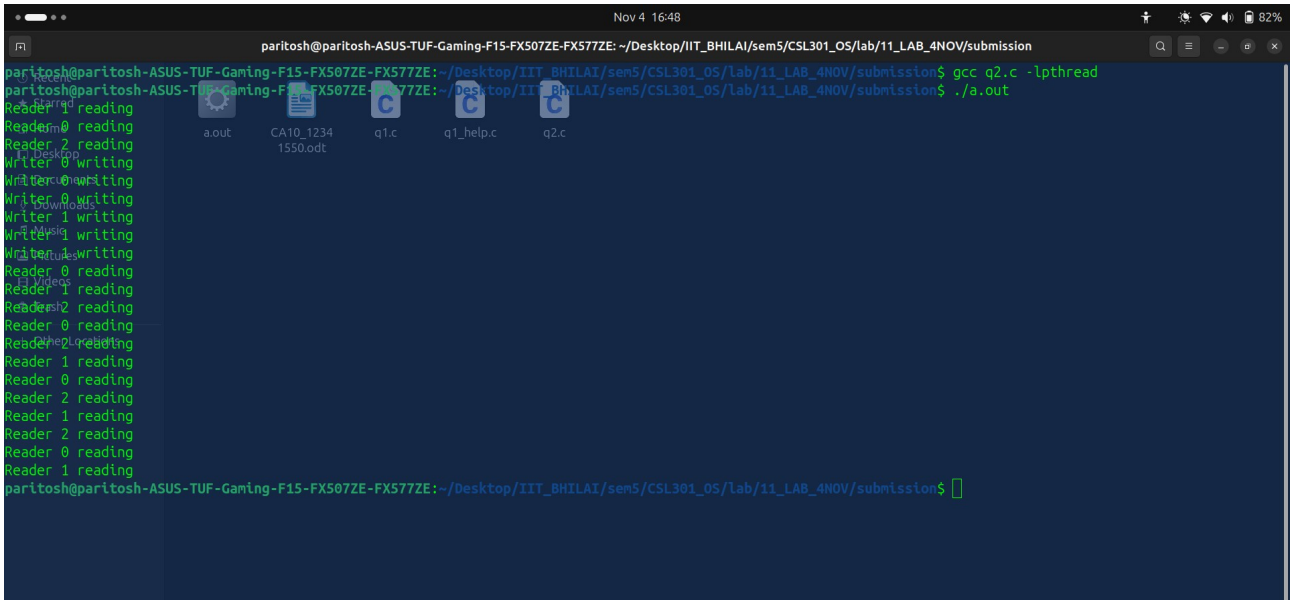
void *reader(void *id) {
    for (int i = 0; i < 5; i++) {
        start_read();
        printf("Reader %ld reading\n", (long)id);
        usleep(100000);
        end_read();
    }
    return NULL;
}

void *writer(void *id) {
    for (int i = 0; i < 3; i++) {
        start_write();
        printf("Writer %ld writing\n", (long)id);
        usleep(150000);
        end_write();
    }
    return NULL;
}

int main() {
    pthread_t r[3], w[2];
    for (long i = 0; i < 3; i++) pthread_create(&r[i], NULL,
reader, (void *)i);
    for (long i = 0; i < 2; i++) pthread_create(&w[i], NULL,
writer, (void *)i);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
```

```
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    return 0;
}
```

output:



Explanation:
Added fairness by blocking readers when writers are active or waiting and by waking writers before readers.

Q3

code:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_LOGS 10

char *log_buffer[MAX_LOGS];
```

```
    int count = 0;


    pthread_mutex_t buffer_lock =
PTHREAD_MUTEX_INITIALIZER;
    pthread_cond_t not_empty =
PTHREAD_COND_INITIALIZER;
    pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;

    void *worker(void *id) {
        for (int i = 0; i < 3; i++) {
            char msg[64];
            sprintf(msg, "Worker %ld message %d", (long)id, i);

            pthread_mutex_lock(&buffer_lock);
            while (count == MAX_LOGS)
                pthread_cond_wait(&not_full, &buffer_lock);

            log_buffer[count++] = strdup(msg);
            printf("Worker %ld queued log. (count=%d)\n", (long)id,
count);

            pthread_cond_signal(&not_empty);
            pthread_mutex_unlock(&buffer_lock);
            usleep(100000);
        }
        return NULL;
    }

    void *logger(void *arg) {
        FILE *f = fopen("log.txt", "w");
        if (!f) {
            perror("fopen");
            return NULL;
        }
```

```
    while (1) {

        pthread_mutex_lock(&buffer_lock);
        while (count == 0)
            pthread_cond_wait(&not_empty, &buffer_lock);

        char *msg = log_buffer[--count];

        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&buffer_lock);

        fprintf(f, "%s\n", msg);
        fflush(f);
        printf("Logger wrote: %s\n", msg);

        free(msg);
        usleep(50000);
    }

    fclose(f);
    return NULL;
}

int main() {
    pthread_t log_thread, workers[3];
    pthread_create(&log_thread, NULL, logger, NULL);
    for (long i = 0; i < 3; i++)
        pthread_create(&workers[i], NULL, worker, (void *)i);

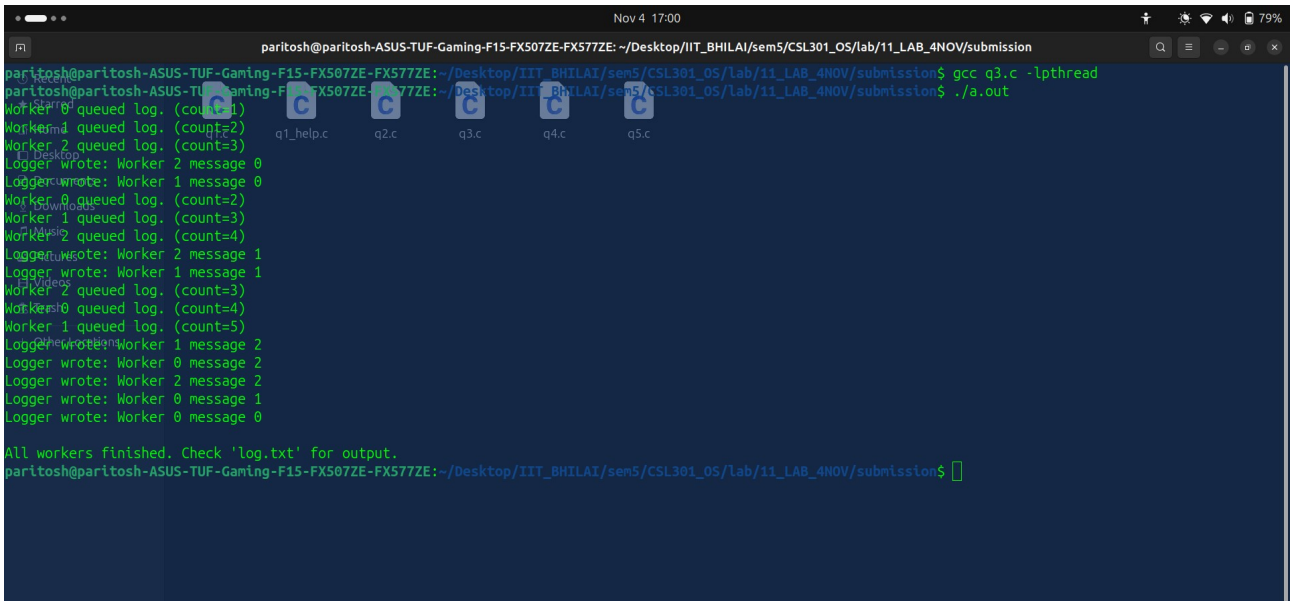
    for (int i = 0; i < 3; i++)
        pthread_join(workers[i], NULL);

    sleep(1);
```

```
    pthread_cancel(log_thread);
    pthread_join(log_thread, NULL);

    printf("\nAll workers finished. Check 'log.txt' for output.\n");
    return 0;
}
```

output:



Explanation:
Guarded the bounded log buffer with mutex/conditions to block producers when full and consumers when empty.

Q4

code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define NTHREADS 3

struct thread_event {
    pthread_mutex_t m;
    pthread_cond_t c;
    int awake;
};

struct thread_event events[NTHREADS];

void *sleeper(void *id) {
    long tid = (long)id;
    pthread_mutex_lock(&events[tid].m);

    while (!events[tid].awake) {
        printf("Thread %ld sleeping...\n", tid);
        //ADD YOUR CODE
        pthread_cond_wait(&events[tid].c, &events[tid].m);
    }

    printf("Thread %ld woke up!\n", tid);

    pthread_mutex_unlock(&events[tid].m);
    return NULL;
}

void *waker(void *arg) {
    sleep(2);

    printf("Waker: waking all threads...\n");


    for (int i = 0; i < NTHREADS; i++) {
        //ADD YOUR CODE
        pthread_mutex_lock(&events[i].m);
```

```
            events[i].awake = 1;
            pthread_cond_signal(&events[i].c);
            pthread_mutex_unlock(&events[i].m);
        }

        return NULL;
    }

    int main() {
        pthread_t t[NTHREADS], w;

        for (int i = 0; i < NTHREADS; i++) {
            events[i].awake = 0;
            pthread_mutex_init(&events[i].m, NULL);
            pthread_cond_init(&events[i].c, NULL);
            pthread_create(&t[i], NULL, sleeper, (void *)(long)i);
        }

        pthread_create(&w, NULL, waker, NULL);
        pthread_join(w, NULL);

        for (int i = 0; i < NTHREADS; i++)
            pthread_join(t[i], NULL);

        printf("All threads finished.\n");
        return 0;
    }
```

output:



Explanation:
Each sleeper waits on its own condition until waker signals under lock, ensuring clean wake-up ordering.

q5

code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_JOBS 5
#define NUM_WORKERS 3

int jobs[MAX_JOBS];
int count = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty =
PTHREAD_COND_INITIALIZER;

void *dispatcher(void *arg) {
    int job_id = 1;
```

```
    while (job_id <= 10) {
        pthread_mutex_lock(&lock);

        while (count == MAX_JOBS)
            pthread_cond_wait(&not_full, &lock);

        jobs[count++] = job_id;
        printf("Dispatcher added job %d (count=%d)\n", job_id,
count);

        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&lock);
        job_id++;

        usleep(100000);
    }
    return NULL;
}

void *worker(void *arg) {
    long id = (long)arg;

    while (1) {
        pthread_mutex_lock(&lock);

    while (count == 0)
            pthread_cond_wait(&not_empty, &lock);

        int job = jobs[--count];
        printf("Worker %ld processing job %d (remaining=%d)\
n", id, job, count);

        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&lock);
```

```
        usleep(200000);
    }
    return NULL;
}

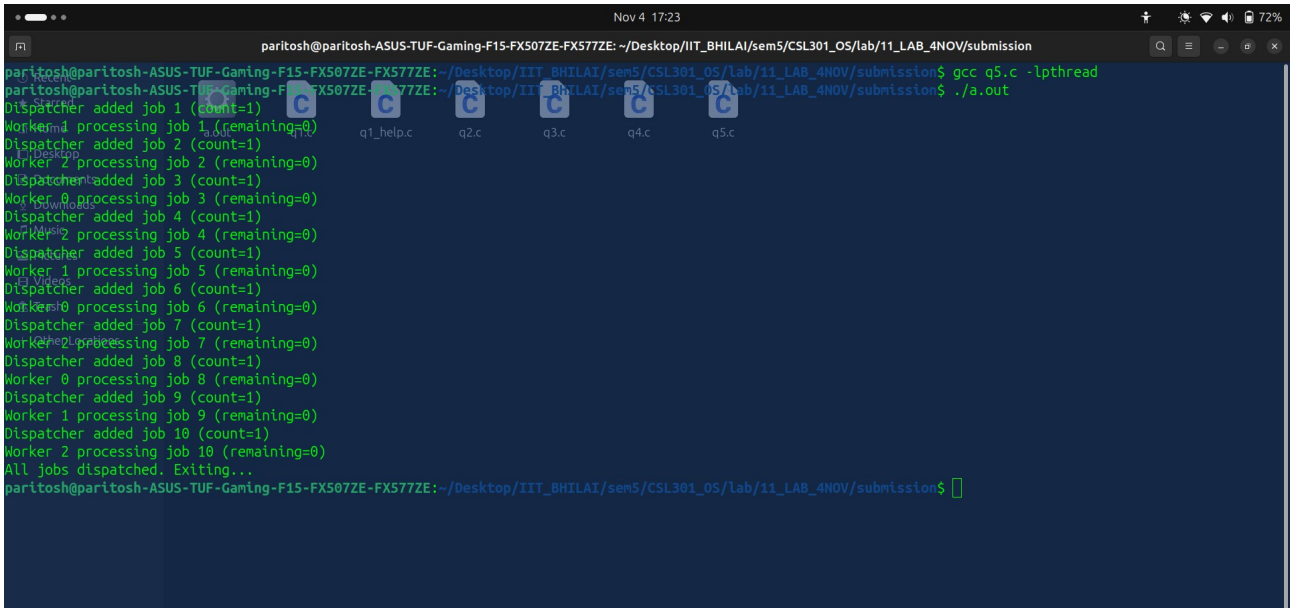
int main() {
    pthread_t disp, workers[NUM_WORKERS];

    pthread_create(&disp, NULL, dispatcher, NULL);
    for (long i = 0; i < NUM_WORKERS; i++)
        pthread_create(&workers[i], NULL, worker, (void *)i);

    pthread_join(disp, NULL);
    sleep(2);

    printf("All jobs dispatched. Exiting...\n");
    return 0;
}
```

output:



Explanation:
Replaced busy polling with pthread_cond_wait/signal, letting dispatcher block when the queue is full and workers block when empty so CPU time isn’t wasted.